Université de Sherbrooke – Département d'informatique

IFT814 – Cryptographie

Martin Fiset, chargé de cours

Devoir 2

Date de remise : vendredi 6 novembre 2026 à 23 h 59 via Turnin

---

Modalités

- **Équipes** d'au plus 4 personnes. Ce devoir compte pour **10 %** de la note finale. Il est noté sur 100 points, plus un bonus facultatif (voir la fin du document).
- **Remise** sur Turnin : un seul fichier PDF par équipe, contenant le numéro de l'équipe, tous les noms et le code source **comme texte** dans le corps du document (pas de fichiers .py joints). Joignez aussi la « Déclaration d'usage » des outils d'IA générative de la Faculté des sciences, même si vous n'avez rien utilisé (écrivez alors « aucun usage »).
- **Fiche de paramètres.** Chaque équipe reçoit une fiche personnelle (Equipe_XX.txt) contenant ses clés, ses cryptogrammes et ses données. Vous devez utiliser **votre** fiche : les traces calculées avec les paramètres d'une autre équipe seront considérées comme incorrectes. Les cryptogrammes et les données longues portent une empreinte qui permet de détecter une erreur de copie.
- **Langage** : Python, C, Java (ou un autre langage, sur approbation préalable). Vous devez écrire vous-mêmes les algorithmes demandés. Sont permis : un générateur aléatoire sûr fourni par le langage (`secrets`, `SecureRandom`, `/dev/urandom`), les fonctions standard sur les chaînes et les entiers, et une bibliothèque de graphiques pour les histogrammes. Sont interdites : les bibliothèques qui implémentent déjà un chiffrement ou un MAC.
- **Intelligence artificielle** : utilisation **limitée** (niveau 1 des balises de la Faculté). Vous pouvez y recourir pour vous inspirer, explorer un sujet ou vous faire expliquer une notion. Il est **interdit** de lui faire produire votre code, vos traces ou vos réponses. Toute utilisation doit être déclarée. Le professeur peut demander à une équipe d'expliquer son code et ses résultats.
- **Présentation** : code commenté et lisible, exemples d'exécution pour chaque algorithme, français soigné, sources citées. Un outil comme Zotero peut vous aider à tenir votre bibliographie. Le code doit être inséré dans le PDF avec une police à chasse fixe.
- **Retard** : À moins d'une entente préalable, chaque jour de retard entraînera une pénalité de 2% sur la note finale du travail.

Conventions

⊕ désigne le OU exclusif bit à bit. pow(a, b, n) désigne l'exponentiation modulaire a^b mod n, à calculer par exponentiation rapide (« carré-et-multiplication ») — pas par multiplication répétée ni via une bibliothèque de chiffrement.

Trois personnages : Alice et Bob communiquent, Ève observe le canal (attaque passive) et Mallory contrôle le canal — elle peut intercepter, modifier et injecter des messages (attaque active). Tous connaissent les algorithmes (principe de Kerckhoffs) mais pas les clés privées.

Toutes les valeurs numériques imposées (p, g, x, y, p, q, e, m, c', m1, m2) sont fournies dans votre fiche de paramètres. Les lignes « [Verification n.n] » donnent le résultat attendu de la sous-question correspondante : reproduisez-le avec votre propre code et incluez-le explicitement dans votre remise.

À la dernière page de ce document, se trouve des liens vers des définitions plus précises sur des termes utilisés dans ce devoir.

Partie 1 – Diffie-Hellman (30 points)

On travaille dans le groupe multiplicatif Z*_p, avec p = 2^61 − 1 (un nombre premier de Mersenne) et un générateur public g = 37, qui est une racine primitive modulo p (son ordre multiplicatif vaut p − 1). Ces deux valeurs sont les mêmes pour toute la classe et figurent dans votre fiche.

1.1 Implémentation du protocole (10 points)

Écrivez Gen (choix d'un exposant privé aléatoire dans {2, …, p−2} à l'aide d'un générateur sûr), la fonction PubKey(x) = g^x mod p, et la fonction Secret (clé_privée, clé_publique_reçue) qui calcule le secret partagé. Faites un exemple d'exécution complet avec des valeurs aléatoires (Alice et Bob choisis par votre code), en vérifiant que les deux parties obtiennent le même secret.

1.2 Trace avec les clés imposées (5 points)

Reprenez les clés privées x (Alice) et y (Bob) imposées par votre fiche. Calculez h_A = g^x mod p et h_B = g^y mod p, puis le secret partagé K = h_B^x mod p = h_A^y mod p. Donnez K tronqué à 32 bits (K mod 2^32). Comparez vos trois résultats aux valeurs « [Verification 1.2] » de votre fiche : ils doivent correspondre exactement.

1.3 Chiffrement du message avec la clé dérivée (5 points)

Le secret K (tronqué à 32 bits, soit 4 octets en représentation « big-endian » ou boutisme en français) sert de clé à un chiffrement par flot simplifié : $c[i] = m[i] \oplus k[i \bmod 4]$, appliqué octet par octet sur l'encodage ASCII du message. Chiffrez le message de votre fiche avec K et donnez le résultat en hexadécimal. Comparez à la valeur « [Verification 1.3] ».

1.4 Espionnage passif : ce qu'Ève peut et ne peut pas faire (5 points)

- Ève observe p, g, h_A et h_B sur le canal, mais ni x ni y. Expliquez précisément quel problème mathématique elle devrait résoudre pour retrouver K, et pourquoi on le croit difficile pour un p de cette taille (61 bits) dans un vrai déploiement — puis pourquoi 61 bits sont en réalité **beaucoup trop petits** pour un usage réel (donnez un ordre de grandeur de l'effort de calcul, et comparez à la taille de p recommandée en pratique, p. ex. 2048 ou 3072 bits).

- Ève ne peut pas non plus déchiffrer le message de 1.3 sans K. Est-ce que la connaissance du texte clair français attendu (contexte, longueur, vocabulaire probable) lui donne un raccourci pour retrouver K sans résoudre le logarithme discret ? Justifiez.

1.5 Attaque active : l'homme du milieu (10 points)

Un échange Diffie-Hellman non authentifié est vulnérable à une attaque active. Mallory intercepte les deux demi-clés publiques et négocie séparément un secret avec Alice et un autre avec Bob, sans qu'aucun des deux ne s'en aperçoive.

- Décrivez l'attaque étape par étape : que reçoit et qu'envoie Mallory à la place d'Alice et de Bob, combien de secrets partagés distincts existent au final, et comment Mallory peut lire (et modifier) tous les messages chiffrés avec le schéma de 1.3 qui transitent ensuite.
- Implémentez une simulation de cette attaque : trois entités logicielles (Alice, Bob, Mallory) échangeant des messages via une fonction qui simule le canal en passant systématiquement par Mallory. Faites la preuve que Mallory déchiffre correctement un message qu'Alice croit envoyer directement à Bob, le modifie, puis le retransmet sans que Bob détecte l'altération.
- Quelle propriété cryptographique, absente du protocole de base, permettrait d'empêcher cette attaque ? Nommez-la et expliquez en une phrase comment elle intervient (implémentation non requise).

Partie 2 – RSA « textbook » (35 points)

Votre fiche impose deux nombres premiers p et q (environ 28 bits chacun, donc N ≈ 56 bits — assez petit pour que tous les calculs restent rapides en Python natif, mais représentatif de la structure d'une vraie clé RSA).

2.1 Génération de la paire de clés (5 points)

- Vérifiez que p et q sont premiers avec un test de primalité de votre choix (p. ex. Miller-Rabin — vous pouvez utiliser une fonction fournie par votre langage, mais expliquez son principe en une phrase). Calculez N = p·q et φ(N) = (p−1)(q−1).
- L'exposant public est fixé à e = 65 537 pour toutes les équipes. Vérifiez que pgcd(e, φ(N)) = 1, puis calculez d = e^{-1} mod φ(N) **avec votre propre implémentation de l'algorithme d'Euclide étendu** (montrez les étapes intermédiaires, comme dans le devoir 1). Comparez d à la valeur « [Verification 2.1] ».

2.2 Chiffrement et déchiffrement (10 points)

- Chiffrez le message m de votre fiche : c = m^e mod N. Comparez à « [Verification 2.2] ».
- Déchiffrez le cryptogramme c' fourni séparément dans votre fiche (ce n'est pas le chiffré de m — c'est un cryptogramme indépendant dont vous ne connaissez pas le clair) : calculez m' = c'^d mod N et comparez à « [Verification 2.2] ».

2.3 Cas limites (5 points)

RSA textbook n'a aucune protection intégrée contre les messages « dégénérés ».

- Que vaut Enc(0) ? Que vaut Enc(1) ? Que vaut Enc(N−1) ? Calculez les trois avec votre implémentation et expliquez pourquoi un attaquant qui voit un de ces cryptogrammes n'a besoin d'aucune cryptanalyse pour reconnaître le clair.
- Plus généralement, RSA textbook est **déterministe** : que peut en déduire un attaquant qui observe deux cryptogrammes identiques envoyés à des moments différents, même sans connaître e, N ou d ?

2.4 Malléabilité (10 points)

RSA textbook est homomorphe pour la multiplication modulo N : Enc(m1)·Enc(m2) mod N = Enc(m1·m2 mod N).

- À partir du cryptogramme c calculé en 2.2 (chiffré de m), calculez c_mal = c · 2^e mod N **sans connaître m**. Comparez à « [Verification 2.4] ».
- Si un oracle de déchiffrement vous donnait Dec(c_mal), que devriez-vous obtenir en fonction de m ? Vérifiez-le avec votre propre clé privée (vous la connaissez ici à des fins pédagogiques ; dans une vraie attaque, seul l'oracle la connaîtrait) et comparez à « [Verification 2.4] ».
- Décrivez un scénario concret où cette propriété permet une attaque réelle (p. ex. un système de facturation qui chiffre un montant en RSA textbook sans padding) : que peut

faire un attaquant qui intercepte un cryptogramme et peut le soumettre à un service de déchiffrement, sans jamais apprendre la clé privée ?

2.5 Attaque par dictionnaire (5 points)

Un attaquant intercepte c = m^e mod N (le cryptogramme de 2.2) et sait que m appartient à un petit ensemble de messages plausibles (p. ex. un montant en dollars entre 0 et 999, ou un code à 4 chiffres).

- Construisez, avec votre code, un ensemble d'au moins 1000 candidats plausibles incluant la vraie valeur de m (vous pouvez tricher légèrement puisque vous connaissez déjà m — le but est de montrer la mécanique de l'attaque). Pour chaque candidat, chiffrez-le et comparez au cryptogramme intercepté c ; identifiez le bon m et montrez le nombre d'essais nécessaires.
- Pourquoi RSA-OAEP (mentionné dans les présentations du cours) rend-il cette attaque impraticable, alors que le chiffrement RSA sous-jacent est le même ? Une phrase suffit.

Partie 3 – Signatures RSA « textbook » (35 points)

On réutilise la paire de clés (N, e, d) de la partie 2. En signature RSA textbook, Sign(sk, m) = m^d mod N et Verify(pk, m, s) accepte si s^e mod N = m.

3.1 Implémentation (10 points)

Écrivez Sign et Verify. Testez avec au moins deux messages de votre choix : signez, vérifiez que la signature est acceptée, puis modifiez le message d'un seul bit et vérifiez que la signature est rejetée.

3.2 Signatures imposées (via un oracle simulé) (5 points)

Votre fiche fournit deux messages m1 et m2. Imaginez que vous êtes un attaquant qui a accès à un **oracle de signature** — vous pouvez lui soumettre n'importe quel message et il vous retourne la signature correspondante, sans jamais révéler d. Simulez cet oracle avec votre propre code (il peut, en coulisses, utiliser d — c'est la clé privée du signataire légitime, pas la vôtre) et interrogez-le pour obtenir s1 = Sign(m1) et s2 = Sign(m2). Comparez à « [Verification 3.2] ».

3.3 Falsification existentielle simple (5 points)

Sans interroger l'oracle et sans connaître d, un attaquant peut tout de même produire une paire (m, s) valide.

- Choisissez une valeur s quelconque (p. ex. aléatoire dans Z*_N) et calculez m = s^e mod N. Vérifiez avec Verify que (m, s) est accepté. Pourquoi cette attaque fonctionne-t-elle toujours, indépendamment de la clé ? Pourquoi est-elle malgré tout peu dangereuse en pratique (que contrôle, et surtout que ne contrôle pas, l'attaquant) ?

3.4 Falsification par message choisi (forgerie multiplicative) (15 points)

Contrairement à 3.3, ici l'attaquant vise un message **cible** de son choix, en exploitant l'homomorphisme multiplicatif des signatures RSA textbook.

- À partir des signatures s1 et s2 obtenues en 3.2, calculez m_cible = m1 · m2 mod N et s_cible = s1 · s2 mod N, **sans jamais appeler l'oracle sur m_cible**. Vérifiez avec Verify que (m_cible, s_cible) est une paire valide. Comparez vos deux valeurs à « [Verification 3.4] ».
- Démontrez algébriquement pourquoi s_cible^e mod N = m_cible (une ou deux lignes de calcul avec les exposants, en utilisant Verify(s1)=m1 et Verify(s2)=m2).
- Supposons que m_cible corresponde, dans une vraie application, à un ordre de transfert bancaire dont le montant est le produit de deux montants anodins déjà signés (m1, m2). Expliquez concrètement pourquoi un système qui signerait des documents avec RSA textbook « nu » (sans fonction de hachage ni padding, comme PSS) serait dangereux à déployer, et nommez le mécanisme standard qui neutralise cette attaque.

Barème sommaire

Section	Contenu	Points
1.1–1.5	Diffie-Hellman : implémentation, trace, chiffrement, espionnage passif, MITM	30
2.1–2.5	RSA textbook : keygen, chiffrement/déchiffrement, cas limites, malléabilité, dictionnaire	35
3.1–3.4	Signatures RSA textbook : implémentation, trace, forgerie existentielle, forgerie par message choisi	35
Total		**100**

Contenu attendu du PDF et info supplémentaire

- Page titre : numéro d'équipe, noms et matricules, Déclaration d'usage des outils d'IA générative.
- Pour chaque question : code (texte), exemples d'exécution, traces et réponses rédigées.
- Une courte section de discussion : limites de vos implémentations et améliorations possibles.
- Bibliographie et sources consultées.
- Boutisme ou Big-endian:
  - Boutisme — Wikipédia
- Nombre de Mersenne premier
  - Nombre de Mersenne premier — Wikipédia
- Test de primalité Miller-Rabin
  - Test de primalité de Miller-Rabin — Wikipédia